충전소 예약 서비스 - 기획 검토 문서

> 작성 목적 : 테이블 담당자별 기획 허점 및 확인 필요 사항 정리
> 각 담당자는 본인 파트를 확인 후 팀 내 공유 요망

📋 목차

1. STATION / CHARGER
2. RESERVATION
3. MEMBER / MEMBER_TOKEN
4. PENALTY_HISTORY
5. INQUIRY
6. ADMIN

---

1. STATION / CHARGER

> 담당 : 최병인 (API 연동 + 두 테이블 관리)

🟤 핵심 이슈 - API 트래픽 제한

오피셜 확인 필요

- 차지인포(ChargeInfo) API 개발계정 하루 트래픽 한도 공식 확인 필요
- 현재 AI 피셜로는 개발계정 **1,000건/일** 제한

트래픽 계산식

```
전국 충전기 약 30만대 기준
300,000 ÷ 1,000(건당 최대) = 300번 API 호출 = 300건 소모

10분마다 상태 스케줄러를 돌린다면
300건 × 144번(하루) = 43,200건/일 → 개발계정 한도 초과
```

현실적인 해결 방향

```
지역 범위를 학원 근처로 한정할 경우
약 200~500대 가정 시
500 ÷ 1,000 = 1번 호출 = 1건 소모
10분마다 → 하루 144건 → 개발계정으로 충분히 감당 가능
```

🟡 동기화 주기 제안

대상	현재 기획	제안	이유

대상	현재 기획	제안	이유
충전소/충전기 기본 정보	24시간	72시간	거의 변하지 않는 정적 데이터
충전기 상태값	10분	5~10분 유지	예약 서비스 핵심 데이터

🟢 캐싱 구조 확인 필요

기본 개념 (오피셜 확인 필요)

```
공공 API → DB 저장 → DB 내부에서 스케줄러 운용
→ 사용자 조회는 DB만 참조
→ API 트래픽 부담 최소화
```

단, 이 구조는 **완전한 실시간이 아닌 준실시간** (최대 10분 지연)
예약 서비스 특성상 허용 가능한 수준인지 팀 내 합의 필요

☑ 체크리스트

- ☐ 차지인포 API 개발계정 트래픽 한도 공식 확인
- ☐ 서비스 대상 지역 범위 확정
- ☐ CHARGER 테이블에 status 컬럼 존재 여부 확인 (AVAILABLE / CHARGING / RESERVED / 점검중 등)
- ☐ 동기화 주기 팀 내 합의

---

2. RESERVATION

> 담당 : 신재욱

🟢 이슈 1 - chargerId 존재 여부 검증 없음

```
현재 : 없는 chargerId로 예약 요청 시 그대로 DB에 저장됨
문제 : 유령 예약 생성 가능
해결 : 서비스 진입 시점에 CHARGER 테이블 조회
       존재하지 않으면 즉시 404 반환
```

🟢 이슈 2 - 충전기 현재 상태 확인 로직 없음

```
현재 : 시간 중복 체크만 존재
문제 : 고장/점검 중인 충전기도 예약 가능
해결 : chargerId 조회 시 status 컬럼도 함께 확인
       AVAILABLE 상태가 아니면 예약 거부
```

🟢 이슈 3 - chargerType 검증 위치 문제

```
현재 : DTO에서 @NotBlank만 체크 (형식 검증만 존재)
문제 : "ABC", "ㅁㄴㅇ" 같은 이상한 값도 통과
해결안 A : 서비스에서 DB의 실제 chargerType과 대조
해결안 B : Request에서 chargerType 필드 제거
           chargerId로 DB 조회해서 타입 직접 가져오기
           (B안이 더 깔끔함)
```

🟢 이슈 4 - PIN 중복 가능성

```
현재 : Math.random()으로 0000~9999 랜덤 생성
문제 : 동시 예약이 몰릴 경우 중복 PIN 발생 가능
       중복 체크 로직 없음
해결 : 생성 후 DB에서 중복 여부 확인
       중복이면 재생성 (최대 3회 시도)
```

🟢 이슈 5 - 패널티 연동 로직 없음

```
현재 : 예약 생성 시 패널티 체크 없음
문제 : 노쇼 누적 회원도 계속 예약 가능
해결 : 예약 생성 시점에 MEMBER 테이블 패널티 만료시간 조회
       만료 전이면 예약 거부

노쇼 감지 (기존 스케줄러 활용)
→ startTime이 지났는데 RESERVED 상태인 예약 감지
→ 유예 시간 몇 분으로 할지 팀 내 합의 필요
→ 노쇼 처리 + PENALTY_HISTORY insert + penaltyCount 증가
```

☑ 체크리스트

- ☐ 이슈 1, 2 서비스 코드 반영 (우선순위 높음)
- ☐ chargerType 처리 방식 팀 내 결정 (A안 vs B안)
- ☐ PIN 중복 체크 로직 추가
- ☐ 패널티 체크 로직 추가
- ☐ 노쇼 유예 시간 기준 합의

---

3. MEMBER / MEMBER_TOKEN

> 담당 : 박은비

🟢 본인인증 방식 미결정

```
이메일 인증
→ 구현 쉬움, 별도 비용 없음, 실시간성 낮음
→ 학습용 프로젝트에 적합
```

```
휴대폰 인증
→ 실시간성 높음
→ SMS API 비용 발생 (coolsms, 네이버 클라우드 등 유료)
```

⚠ **이 결정이 안 나면 MEMBER 테이블 컬럼 설계 확정 불가**
빠른 결정 필요

◍ JWT 2트랙 관련 확인 사항

```
액세스 토큰 + 리프레시 토큰 방식 채택은 맞는 방향
확인 필요 사항
- 토큰 만료 시 자동 삭제 로직 있는지
- 로그아웃 시 토큰 무효화 처리 있는지
- 리프레시 토큰 탈취 대비 rotation 정책 있는지
```

◍ RESERVATION과 연결되는 케이스

```
회원 탈퇴 시 진행 중인 예약 처리 정책 필요
→ 예약 자동 취소 처리할지
→ 현재 예약 서비스 코드에 해당 케이스 없음
```

☑ 체크리스트

- ☐ 본인인증 방식 확정 (팀 전체 합의)
- ☐ 토큰 만료/무효화/rotation 정책 확인
- ☐ 탈퇴 회원 예약 처리 정책 결정

---

4. PENALTY_HISTORY

> 담당 : 박지환

☑ 확정된 방향

```
법령 기반 주차시간 감지 → 현실적으로 구현 불가, 제외
패널티 부과 기준 → 노쇼에 한해서만 책정
```

◍ 세부 정책 미확정 항목

```
노쇼 감지 기준
→ startTime 기준 몇 분 유예할지 미결정

패널티 부과 정책
```

```
→ 1회 노쇼 = 24시간 예약 불가 (1차 목표)
→ 누적 시 점진적 증가 로직은 추후 고도화로 분리

패널티 만료시간 저장 위치
→ MEMBER 테이블에 둘지
→ PENALTY_HISTORY 테이블에 둘지 결정 필요
```

🟤 테이블 구조 확인 필요

```
PENALTY_HISTORY 컬럼 확인
- 패널티 발생 일시
- 연관 예약 ID (RESERVATION 참조)
- 패널티 만료 일시
- 패널티 사유 (현재는 노쇼 단일)

MEMBER 테이블 확인
- penaltyCount 컬럼 존재 여부
- 패널티 만료시간 컬럼 존재 여부
```

☑ 체크리스트

- ☐ 노쇼 유예 시간 기준 확정
- ☐ 패널티 만료시간 저장 위치 결정
- ☐ PENALTY_HISTORY 테이블 컬럼 설계 확정
- ☐ MEMBER penaltyCount 컬럼 확인

---

5. INQUIRY

> 담당 : 김민수

☑ 확정된 방향

```
단순 게시판형 + ADMIN 답변 구조
복잡한 실시간 채팅은 후순위
```

⚠ 웹소켓 관련 주의사항

```
현재 프로젝트에 웹소켓이 이미 구현되어 있음
기술적으로 1:1 채팅 구현은 가능하나

담당자 코딩 레벨에 따라 난이도가 크게 올라감
무리하게 채팅 방식으로 가지 말고
게시판형으로 먼저 완성 후 여유 있을 때 고도화 권장
```

🟢 기본 구조 제안

```
INQUIRY 테이블 컬럼
- 제목 / 내용
- 카테고리 (예약문의 / 패널티이의 / 계정문의 / 기타)
- 상태 (답변대기 / 답변완료)
- 예약번호 참조 컬럼 (선택, RESERVATION 연동 시 처리 빠름)
- 작성일시 / 답변일시
```

🟢 RESERVATION 연동 고려

```
예약 관련 문의 시 예약번호 첨부 가능하면
ADMIN이 바로 조회 가능해서 처리 속도 향상
필수는 아니지만 UX 측면에서 권장
```

☑ 체크리스트

- ☐ 게시판형으로 방향 확정 (팀 합의)
- ☐ INQUIRY 테이블 컬럼 설계 확정
- ☐ RESERVATION 연동 여부 결정
- ☐ 채팅 방식은 추후 고도화로 분리

---

6. ADMIN

> 담당 : 추상현

🟢 ADMIN이 관리해야 할 범위

```
RESERVATION 관련
- 전체 예약 조회 / 강제 취소
- 노쇼 처리 내역 확인

MEMBER 관련
- 회원 목록 조회
- 패널티 부과 / 해제 권한

PENALTY_HISTORY 관련
- 패널티 이력 조회
- 이의제기 처리 (INQUIRY 연동)

STATION / CHARGER 관련
- 충전기 상태 수동 변경 (고장 처리 등)
- API 동기화 수동 트리거

INQUIRY 관련
```

```
- 문의 목록 조회
- 답변 작성
```

🟢 ADMIN 계정 보안

```
일반 회원과 완전히 분리된 인증 체계 필요
ADMIN 전용 엔드포인트 접근 제한 확인
JWT 토큰에 role 구분 있는지 확인
```

☑ 체크리스트

- ☐ ADMIN 담당자 확정
- ☐ ADMIN 관리 범위 팀 내 합의
- ☐ ADMIN 전용 인증 체계 확인
- ☐ 각 테이블 ADMIN 조회/수정 API 구현 계획

📌 전체 우선순위 요약

우선순위	항목	담당
🔴 즉시	차지인포 API 트래픽 한도 공식 확인	CHARGER 담당
🔴 즉시	서비스 대상 지역 범위 확정	팀 전체
🔴 즉시	본인인증 방식 확정	팀 전체
🔴 즉시	chargerId 존재/상태 검증 로직 추가	RESERVATION 담당
🟡 단기	노쇼 유예 시간 기준 합의	팀 전체
🟡 단기	패널티 만료시간 저장 위치 결정	PENALTY 담당
🟡 단기	chargerType 처리 방식 결정	RESERVATION 담당
🟢 여유	INQUIRY 채팅 고도화	INQUIRY 담당
🟢 여유	패널티 점진적 증가 로직	PENALTY 담당